



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №8
із дисципліни «Технології розробки програмного забезпечення»
Тема: «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR»

Виконав:
Студент групи ІА-24
Зелінський І.О.

Перевірив:
Мякий М. Ю.

Зміст

Тема.....	3
Мета.....	3
Завдання.....	3
Теоретичні відомості.....	3
Обрана тема.....	5
Інтерфейс Visitable.....	5
Інтерфейс StatisticsVisitor.....	5
Клас StatisticsCollector.....	6
Реалізація Visitable для моделей.....	7
Тестування.....	7
Висновок.....	8

Тема: Шаблони «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR»

Мета: Навчитися реалізовувати шаблони «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR»

Завдання:

1. Ознайомитися з короткими теоретичними відомостями.
2. Реалізувати частину функціоналу робочої програми у вигляді класів і їх взаємодій для досягнення конкретних функціональних можливостей.
3. Застосування одного з розглянутих шаблонів при реалізації програми.

Теоретичні відомості

Шаблони проєктування

Шаблон проєктування — це типовий підхід до вирішення поширених проблем, які виникають під час створення архітектури програмного забезпечення. Вони не є готовими бібліотеками чи функціями, які можна просто скопіювати в проєкт. Замість цього шаблон описує загальний принцип, який потребує адаптації до конкретних умов програми.

Найпростіші шаблони, які мають сенс лише в певній мові програмування, називають *ідіомами*. Найбільш універсальні — це *архітектурні шаблони*, які можна застосувати в будь-якій мові програмування. Їх використовують для створення структури програми в цілому, а не окремих її частин.

Класифікація шаблонів

Шаблони можна поділити на три основні групи:

1. **Породжуючі:** допомагають створювати об'єкти, зменшуючи кількість залежностей у програмі.
2. **Структурні:** визначають, як об'єкти можуть взаємодіяти між собою.
3. **Поведінкові:** оптимізують обмін даними та комунікацію між об'єктами.

Шаблон «Composite»

Шаблон використовується для складання об'єктів в деревоподібну структуру для подання ієрархій типу «частина цілого». Даний шаблон дозволяє уніфіковано обробляти як поодинокі об'єкти, так і об'єкти з вкладеністю.

Простим прикладом може служити складання компонентів всередині звичайної форми. Форма може містити дочірні елементи (поля для введення тексту, цифр, написи, малюнки тощо); дочірні елементи можуть в свою чергу містити інші елементи. Наприклад, при виконанні операції розтягування форми необхідно, щоб вся ієрархія розтягнулася відповідним чином. В такому випадку форма розглядається як композитний об'єкт і операція розтягування застосовується до всіх дочірніх елементів рекурсивно. Даний шаблон зручно використовувати при необхідності подання та обробки ієрархій об'єктів.

Шаблон «Flyweight»

Шаблон використовується для зменшення кількості об'єктів в додатку шляхом поділу цих об'єктів між ділянками додатку. Flyweight являє собою поділюваний об'єкт. Дуже важливою є концепція «внутрішнього» і «зовнішнього» станів. Внутрішній стан відображає дані, характерні саме поділюваному об'єкту (наприклад, код букви); зовнішній стан несе інформацію про його застосування в додатку (наприклад, рядок і стовпчик). Внутрішній стан зберігається в самому поділюваному об'єкті, зовнішній - в об'єктах додатку (контексту використання поділюваного об'єкта).

Шаблон «Interpreter»

Даний шаблон використовується для подання граматики і інтерпретатора для вибраної мови (наприклад, скриптової). Граматика мови представлена термінальними і нетермінальними символами, кожен з яких інтерпретується в контексті використання. Клієнт передає контекст і сформовану пропозицію в використовувану мову в термінах абстрактного синтаксичного дерева (деревоподібна структура, яка однозначно визначає ієрархію виклику підвиразів), кожен вираз інтерпретується окремо з використанням контексту. У разі наявності дочірніх виразів, батьківський вираз інтерпретує спочатку дочірні (рекурсивно), а потім обчислює результат власної операції.

Шаблон «Visitor»

Шаблон відвідувач дозволяє вказувати операції над елементами без зміни структури конкретних елементів. Таким чином вкрай зручно додавати нові операції, проте дуже важко додавати нові елементи в ієрархію (необхідно додавати відповідні методи для обробки їх відвідувань в кожного відвідувача).

Даний шаблон дозволяє групувати однотипні операції, що застосовуються над різнотипними об'єктами.

Обрана тема: Online radio station (iterator, adapter, factory method, facade, visitor, client-server)

Додаток повинен служити сервером для радіостанції з можливістю мовлення на радіостанцію (64, 92, 128, 196, 224 kb/s) в потоковому режимі; вести облік підключених користувачів і статистику відвідувань і прослуховувань; налаштувати папки з піснями і можливість вести списки програвання або playlists (не відтворювати всі пісні).

Хід роботи

Під час розробки програми я реалізував шаблон "Visitor ". Для своїх основних класів User, Song і Playlist я реалізував цей патерн, що буде використовуватися для виконання аналізу даних, наприклад, підрахунку кількості пісень, кількості плейлистів користувача і загальної кількості виконавців.

Інтерфейс Visitable (Описує об'єкт, який може бути відвіданий відвідувачем (класом, що реалізує StatisticsVisitor). Він містить метод accept, який приймає візита як аргумент):

```
package com.OnlineRadio.OnlineRadioStation.visitor;  
  
public interface Visitable {  
    void accept(StatisticsVisitor visitor);  
}
```

Рисунок 1. Інтерфейс Visitable

Інтерфейс StatisticsVisitor (Описує інтерфейс для відвідувачів, які можуть відвідувати різні типи об'єктів у системі (користувачі, пісні, плейлисти). Він містить методи visitUser, visitSong і visitPlaylist.):

```
package com.OnlineRadio.OnlineRadioStation.visitor;  
  
import com.OnlineRadio.OnlineRadioStation.models.User;  
import com.OnlineRadio.OnlineRadioStation.models.Song;  
import com.OnlineRadio.OnlineRadioStation.models.Playlist;
```

```
public interface StatisticsVisitor {
    void visitUser(User user);
    void visitSong(Song song);
    void visitPlaylist(Playlist playlist);
}
```

Рисунок 2. Інтерфейс StatisticsVisitor

Клас StatisticsCollector (Реалізує інтерфейс StatisticsVisitor. Він збирає статистику по користувачах, піснях і плейлистах. Має методи visitUser, visitSong і visitPlaylist, які збільшують відповідні лічильники (totalSongs, totalPlaylists, totalArtists) під час обробки кожного об'єкта):

```
package com.OnlineRadio.OnlineRadioStation.visitor;

import com.OnlineRadio.OnlineRadioStation.models.User;
import com.OnlineRadio.OnlineRadioStation.models.Song;
import com.OnlineRadio.OnlineRadioStation.models.Playlist;

public class StatisticsCollector implements StatisticsVisitor {

    private int totalSongs = 0;
    private int totalPlaylists = 0;
    private int totalArtists = 0;

    @Override
    public void visitUser(User user) {
        System.out.println("Processing user: " + user.getLogin());
        totalPlaylists += user.getPlaylistIds().size();
    }

    @Override
    public void visitSong(Song song) {
        System.out.println("Processing song: " + song.getTitle());
        totalArtists += song.getArtistIds().size();
        totalSongs++;
    }

    @Override
    public void visitPlaylist(Playlist playlist) {
        System.out.println("Processing playlist: " + playlist.getName());
    }

    public int getTotalSongs() {
        return totalSongs;
    }

    public int getTotalPlaylists() {
        return totalPlaylists;
    }

    public int getTotalArtists() {
        return totalArtists;
    }
}
```

Рисунок 3. Клас StatisticsCollector

Реалізація Visitable для моделей (наприклад, клас User): Описує метод асерт, який приймає візита і дозволяє йому "відвідувати" об'єкт. Клас User реалізує інтерфейс Visitable, що дозволяє візитору збирати статистику по користувачу:

```
package com.OnlineRadio.OnlineRadioStation.models;

import java.util.List;
import java.util.ArrayList;
import com.OnlineRadio.OnlineRadioStation.visitor.StatisticsVisitor;
import com.OnlineRadio.OnlineRadioStation.visitor.Visitable;

public class User implements Visitable {
    private String id;
    private String login;
    private String password;
    private String ipAddress;
    private Role role;
    private List<String> statisticIds;
    private List<String> playlistIds;

    public User(String id, String login, String password, String
ipAddress, Role role) {
        this.id = id;
        this.login = login;
        this.password = password;
        this.ipAddress = ipAddress;
        this.role = role;
        this.statisticIds = new ArrayList<>();
        this.playlistIds = new ArrayList<>();
    }

    public User(String id, String login, String password, String
ipAddress) {
        this.id = id;
        this.login = login;
        this.password = password;
        this.ipAddress = ipAddress;
        this.statisticIds = new ArrayList<>();
        this.playlistIds = new ArrayList<>();
    }

    @Override
    public void accept(StatisticsVisitor visitor) {
        visitor.visitUser(this);
    }
}
```

Рисунок 4. Клас User

Тестування (Тестування: Клас VisitorPatternTest тестує роботу StatisticsCollector на прикладах користувача, пісень і плейлистів, перевіряючи правильність роботи методів збору статистики):

```
package com.OnlineRadio.OnlineRadioStation.visitor;

import com.OnlineRadio.OnlineRadioStation.models.Playlist;
import com.OnlineRadio.OnlineRadioStation.models.Song;
import com.OnlineRadio.OnlineRadioStation.models.User;
```

```

import org.junit.jupiter.api.Test;

import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;

class VisitorPatternTest {

    @Test
    void testStatisticsCollector() {
        User user = new User("1", "User 1", "password", "192.168.0.1");
        user.addPlaylistId("101");
        user.addPlaylistId("102");

        Song song1 = new Song("201", "Song A", "/music/songA.mp3");
        song1.addArtistId("301");
        song1.addArtistId("302");

        Song song2 = new Song("202", "Song B", "/music/songB.mp3");
        song2.addArtistId("303");

        Playlist playlist = new Playlist("101", "Chill Vibes",
user.getId(), Playlist.Status.PUBLIC);
        playlist.addSongId(song1.getId());
        playlist.addSongId(song2.getId());

        List<Object> items = List.of(user, song1, song2, playlist);

        StatisticsCollector collector = new StatisticsCollector();

        items.forEach(item -> {
            if (item instanceof User) {
                ((User) item).accept(collector);
            } else if (item instanceof Song) {
                ((Song) item).accept(collector);
            } else if (item instanceof Playlist) {
                ((Playlist) item).accept(collector);
            }
        });

        assertEquals(2, collector.getTotalSongs(), "Total songs should be
2");
        assertEquals(2, collector.getTotalPlaylists(), "Total playlists
should be 2");
        assertEquals(3, collector.getTotalArtists(), "Total artists should
be 3");
    }
}

```

Рисунок 5. Інтерфейс Visitable

Висновок: На цій лаб. роботі я зрозумів суть шаблонів «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR», реалізував один з них (VISITOR) для свого проекту “Online radio station”, завдяки цьому шаблону вдалося легко додати нову

функціональність, не змінюючи базовий код об'єктів, що обробляються. Я створив кілька класів відвідувачів, кожен з яких відповідає за обробку певних елементів програми, таких як управління плейлистами, налаштування користувацьких профілів та відображення інформації про пісні.